Gabarito Comentado — Complexidade Algorítmica

Exercício 1 — Laços Sequenciais

```java
public void metodo1(int n) {
    int soma = 0;

    for (int i = 0; i < n; i++) {
        soma += i;
    }

    for (int j = 0; j < n; j++) {
        soma -= j;
    }

    System.out.println(soma);
}
```

Passo 1 — Identificar os blocos

Inicialização da variável
custo constante → **O(1)**

Primeiro `for`
executa **n vezes**

Segundo `for`
executa **n vezes**

`println`
custo constante → **O(1)**

Passo 2 — Montar a função de custo

Se cada operação interna for constante:

```
T(n) = O(1) + n·O(1) + n·O(1) + O(1)
```

Passo 3 — Simplificação

```
T(n) = O(1) + O(n) + O(n) + O(1)
T(n) = 2·O(n) + c
```

Passo 4 — Complexidade assintótica

Constantes são descartadas:

```
O(n)
```

✅ **Resposta final:**
Complexidade temporal **O(n)**

Exercício 2 — Inversão de Array

```java
public int[] inverterArray(int[] arr) {
    int n = arr.length;
    int[] invertido = new int[n];

    for (int i = 0; i < n; i++) {
        invertido[i] = arr[n - 1 - i];
    }

    return invertido;
}
```

Complexidade Temporal

O `for` percorre todo o array:

```
n iterações
```

Cada iteração tem operações constantes.

```
T(n) = n · O(1)
```

Portanto:

```
O(n)
```

Complexidade Espacial

O algoritmo cria:

```
int[] invertido = new int[n];
```

Logo utiliza memória proporcional ao tamanho do array.

```
O(n)
```

✅ **Resposta final**

Tipo	Complexidade
Temporal	O(n)
Espacial	O(n)

Exercício 3 — Busca Linear

```java
public boolean contemElemento(List<Integer> lista, int alvo) {
    for (int num : lista) {
        if (num == alvo) return true;
    }
    return false;
}
```

Melhor Caso

O elemento está na **primeira posição**.

Número de operações:

```
1 comparação
```

Complexidade:

```
O(1)
```

Pior Caso

O elemento:

- está na última posição
  ou
- não existe na lista

O algoritmo percorre todos os elementos.

```
n comparações
```

Complexidade:

```
O(n)
```

Caso Médio

Em média o elemento é encontrado na **metade da lista**.

```
n/2 comparações
```

Assintoticamente:

```
O(n)
```

✅ **Resumo**

Caso	Complexidade
Melhor	O(1)
Médio	O(n)
Pior	O(n)

Exercício 4 — Impressão de Matriz

```java
public void imprimirMatriz(int[][] matriz) {
    for (int i = 0; i < matriz.length; i++) {
        for (int j = 0; j < matriz[i].length; j++) {
            System.out.print(matriz[i][j] + " ");
        }
        System.out.println();
    }
}
```

Matriz n × n

Loop externo:

```
n vezes
```

Loop interno:

```
n vezes
```

Total de execuções:

```
n × n
```

Complexidade:

```
O(n²)
```

Matriz n × m

Loop externo:

```
n
```

Loop interno:

```
m
```

Total:

```
n × m
```

Complexidade:

```
O(n·m)
```

Complexidade Espacial

Nenhuma estrutura adicional relevante é criada.

```
O(1)
```

Exercício 5 — Comparação de Algoritmos

Método A — Busca Linear

```java
public boolean estaContido(int[] arr, int alvo) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == alvo) return true;
    }
    return false;
}
```

Percorre todo o array no pior caso.

```
O(n)
```

Método B — Busca Binária

```java
Arrays.binarySearch(arr, alvo)
```

A cada passo o algoritmo divide o espaço de busca por **2**.

Exemplo:

```
n
n/2
n/4
n/8
...
```

Número de passos:

$\log_2 n$

Complexidade:

```
O(log n)
```

Comparação

Método	Complexidade
Busca Linear	O(n)
Busca Binária	O(log n)

Observação importante

Busca binária **exige array ordenado**.

Ordenar custa:

```
O(n log n)
```

Logo, se o array não estiver ordenado:

```
Ordenação + busca = O(n log n)
```

Exercício 6 — Laço Aninhado

```java
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + " " + j);
    }
}
```

Loop externo:

```
n
```

Loop interno:

```
n
```

Total de execuções:

```
n × n
```

Complexidade:

```
O(n²)
```

Exercício 7 — Crescimento Logarítmico

```
int i = 1;
while (i < n) {
    i = i * 2;
}
```

Valores de `i`:

```
1
2
4
8
16
...
```

O número de multiplicações necessárias para chegar a `n` é:

```
log₂ n
```

Complexidade:

```
O(log n)
```

Exercício 8 — Combinação de Complexidades

Primeiro bloco:

```
for i → n
```

Complexidade:

```
O(n)
```

Segundo bloco:

```
for i → n
   for j → n
```

Complexidade:

```
O(n²)
```

Total:

```
O(n) + O(n²)
```

A maior ordem domina:

```
O(n²)
```

Exercício 9 — Divisão sucessiva

```
while (n > 1) {
    n = n / 2;
}
```

Sequência:

```
n
n/2
n/4
n/8
...
```

Número de passos:

```
log₂ n
```

Complexidade:

```
O(log n)
```

Exercício 10 — Mistura de complexidades

```
for (int i = 0; i < n; i++) {

    int j = 1;

    while (j < n) {
        j = j * 2;
    }
}
```

Loop interno

Multiplica por 2:

```
log n
```

Loop externo

Executa:

```
n vezes
```

Complexidade total

```
n × log n
```

Resultado:

```
O(n log n)
```

Exercício Desafio

```
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        System.out.println(i + j);
    }
}
```

Número de execuções:

```
n + (n-1) + (n-2) + ... + 1
```

Isso forma uma **progressão aritmética**:

```
n(n+1)/2
```

Assintoticamente:

```
O(n²)
```

✅ **Resultado final**

Exercício	Complexidade
1	O(n)
2	O(n) tempo / O(n) espaço
3	O(1) / O(n)
4	$O(n^2)$ ou O(nm)
5	O(n) vs O(log n)
6	$O(n^2)$
7	O(log n)
8	$O(n^2)$
9	O(log n)
10	O(n log n)
Desafio	$O(n^2)$